

HP3M Reasoning Engine — Panduan Integrasi Next.js

Base URL: `https://reasoning.putra-portfolio.cloud`

Dokumentasi Swagger: </docs>

Versi: v1.0.0

Dokumen ini ditujukan khusus untuk **tim programmer web (Next.js / TypeScript)** yang ingin mengintegrasikan aplikasinya dengan layanan AI Reasoning Engine.

Reasoning Engine adalah microservice yang menerima data asesmen, memprosesnya menggunakan AI, dan mengembalikan hasil analisis berupa kekuatan/kelemahan, peluang, dan rencana aksi.

Autentikasi

Semua request ke endpoint `/analyze` memerlukan API Key yang dikirim melalui HTTP Header.

Header	Value
X-API-Key	KunciRahasiaHp3m

[!CAUTION] Jangan pernah expose API Key ke sisi client (browser). Selalu panggil endpoint ini dari sisi server menggunakan Server Action atau Route Handler Next.js.

Endpoint API

GET `/api/v1/health` — Cek Status Server

Tidak perlu autentikasi. Digunakan untuk memastikan server aktif.

```
GET https://reasoning.putra-portfolio.cloud/api/v1/health
```

Response:

```
{ "status": "ok", "service": "hp3m-reasoning-engine" }
```

POST /api/v1/reasoning/analyze — Analisis Satu Item

Menganalisis **satu** sub-dimensi asesmen.

POST <https://reasoning.putra-portfolio.cloud/api/v1/reasoning/analyze>
Headers: X-API-Key: KunciRahasiaHp3m

Request Body:

```
{
  "provider": {
    "name": "openai",
    "model": "gemma4-31b",
    "endpoint": "https://api.regolo.ai/v1",
    "api_key": "sk-xxxxxxx"
  },
  "assessment": {
    "dimension": "Technology",
    "sub_dimension": "Information Capability",
    "final_result": 2.0,
    "expected_level": 3.0,
    "assessment_note": "Dashboard proses sudah ada namun belum ada prediksi berbasis AI."
  }
}
```

POST /api/v1/reasoning/analyze/batch — Analisis Banyak Item Sekaligus 🌟

Endpoint utama yang paling sering digunakan. Mengirimkan **banyak** data asesmen dalam satu request dan memprosesnya secara paralel.

POST <https://reasoning.putra-portfolio.cloud/api/v1/reasoning/analyze/batch>
Headers: X-API-Key: KunciRahasiaHp3m

Request Body:

```
{
  "provider": {
    "name": "openai",
    "model": "gemma4-31b",
    "endpoint": "https://api.regolo.ai/v1",
    "api_key": "sk-xxxxxxx"
  },
```

```

"assessments": [
  {
    "dimension": "Technology",
    "sub_dimension": "Information Capability",
    "final_result": 2.0,
    "expected_level": 3.0,
    "assessment_note": "Dashboard tersedia tapi belum ada prediksi AI"
  },
  {
    "dimension": "Technology",
    "sub_dimension": "Tooling",
    "final_result": 2.0,
    "expected_level": 4.0,
    "assessment_note": "Hanya digunakan untuk visualisasi dasar"
  }
]
}

```

Response — Array dengan urutan yang **sama persis** dengan `assessments` yang dikirim:

```

[
  {
    "classification": "Gap",
    "strength_weakness": "Organisasi memiliki fondasi dashboard yang baik namun kemampuan",
    "opportunity_analysis": [
      "Integrasi modul machine learning ke platform Process Mining yang ada",
      "Pelatihan tim IT terhadap tools AI/ML"
    ],
    "action_plan": [
      "Evaluasi tools PM terhadap dukungan AI (Q1)",
      "Pilot satu use-case prediksi prioritas tinggi (Q2)",
      "Susun roadmap integrasi AI 12 bulan (Q3)"
    ],
    "meta": {
      "gap": 1.0,
      "priority": "Medium",
      "risk_score": 1.5,
      "is_strength": false,
      "is_weakness": true
    }
  }
]

```

Keterangan Field Response:

Field	Tipe	Keterangan
<code>classification</code>	<code>string</code>	"Strength", "Gap", "Near", atau "Mixed"
<code>strength_weakness</code>	<code>string</code>	Narasi AI tentang kondisi saat ini

<code>opportunity_analysis</code>	<code>string[]</code>	Daftar peluang perbaikan
<code>action_plan</code>	<code>string[]</code>	Langkah-langkah aksi yang direkomendasikan
<code>meta.gap</code>	<code>number</code>	Selisih antara <code>final_result</code> dan <code>expected_level</code>
<code>meta.priority</code>	<code>string</code>	"Low" , "Medium" , atau "High"

Cara Integrasi di Proyek Next.js

Ikuti langkah-langkah berikut secara berurutan.

Langkah 1: Setup Environment Variable

Buat atau edit file `.env.local` di root proyek Next.js Anda:

```
# .env.local
REASONING_ENGINE_URL=https://reasoning.putra-portfolio.cloud
REASONING_ENGINE_API_KEY=KunciRahasiaHp3m

# API Key untuk provider LLM (Regolo/OpenAI)
AI_PROVIDER_API_KEY=sk-xxxxxxx
```

[!IMPORTANT] **Jangan** tambahkan `NEXT_PUBLIC_` di depan nama variabel tersebut. Variabel `NEXT_PUBLIC_` akan terbundle ke dalam JavaScript browser dan bisa dibaca oleh siapapun. Variabel di atas hanya boleh diakses dari sisi server (Server Actions, Route Handlers).

Langkah 2: Definisi TypeScript Types

Buat file `types/reasoning-engine.ts` agar kode lebih aman dan autocomplete berfungsi:

```
// types/reasoning-engine.ts

export interface ProviderConfig {
  name: 'openai' | 'ollama' | 'anthropic';
  model: string;
  endpoint: string;
  api_key?: string;
  temperature?: number; // default: 0.2
  max_tokens?: number; // default: 4096
}
```

```

export interface Assessment {
  dimension: string;
  sub_dimension: string;
  final_result: number;
  expected_level: number;
  assessment_note?: string;
  level_definitions?: Record<string, string>;
}

export interface AnalysisMeta {
  gap: number;
  priority: 'Low' | 'Medium' | 'High';
  risk_score: number;
  is_strength: boolean;
  is_weakness: boolean;
}

export interface AnalysisResult {
  classification: string;
  strength_weakness: string;
  opportunity_analysis: string[];
  action_plan: string[];
  meta: AnalysisMeta;
}

```

Langkah 3: Utility Library

Buat file `lib/reasoning-engine.ts` sebagai satu-satunya tempat komunikasi dengan API:

```

// lib/reasoning-engine.ts
import type { Assessment, AnalysisResult, ProviderConfig } from '@types/reasoning-engine'

const BASE_URL = process.env.REASONING_ENGINE_URL!;
const API_KEY = process.env.REASONING_ENGINE_API_KEY!;

// Provider default – ganti model/endpoint sesuai kebutuhan
const DEFAULT_PROVIDER: ProviderConfig = {
  name: 'openai',
  model: 'gemma4-31b',
  endpoint: 'https://api.regolo.ai/v1',
  api_key: process.env.AI_PROVIDER_API_KEY,
  temperature: 0.2,
};

function getHeaders(): HeadersInit {
  return {
    'Content-Type': 'application/json',
    'X-API-Key': API_KEY,
  };
}

```

```

/** Cek apakah Reasoning Engine aktif */
export async function checkHealth(): Promise<{ status: string; service: string }> {
  const res = await fetch(`${BASE_URL}/api/v1/health`);
  if (!res.ok) throw new Error(`Health check failed: ${res.status}`);
  return res.json();
}

/** Analisis satu sub-dimensi */
export async function analyzeSingle(
  assessment: Assessment,
  provider: ProviderConfig = DEFAULT_PROVIDER
): Promise<AnalysisResult> {
  const res = await fetch(`${BASE_URL}/api/v1/reasoning/analyze`, {
    method: 'POST',
    headers: getHeaders(),
    body: JSON.stringify({ provider, assessment }),
  });

  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(`Analyze error ${res.status}: ${JSON.stringify(err)}`);
  }
  return res.json();
}

/**
 * Analisis banyak sub-dimensi sekaligus (Batch).
 * Hasil dikembalikan dalam urutan yang SAMA dengan array input.
 */
export async function analyzeBatch(
  assessments: Assessment[],
  provider: ProviderConfig = DEFAULT_PROVIDER
): Promise<AnalysisResult[]> {
  const res = await fetch(`${BASE_URL}/api/v1/reasoning/analyze/batch`, {
    method: 'POST',
    headers: getHeaders(),
    body: JSON.stringify({ provider, assessments }),
  });

  if (!res.ok) {
    const err = await res.json().catch(() => ({}));
    throw new Error(`Batch analyze error ${res.status}: ${JSON.stringify(err)}`);
  }
  return res.json();
}

```

Langkah 4: Server Action (Cara yang Direkomendasikan)

Server Actions berjalan 100% di server — API Key tidak pernah menyentuh browser:

```
// app/actions/analyze.ts
'use server';

import { analyzeBatch } from '@lib/reasoning-engine';
import type { Assessment, AnalysisResult } from '@types/reasoning-engine';

type AnalyzeResponse =
  | { success: true; results: AnalysisResult[] }
  | { success: false; error: string };

export async function runAnalysis(assessments: Assessment[]): Promise<AnalyzeResponse> {
  try {
    const results = await analyzeBatch(assessments);
    return { success: true, results };
  } catch (err) {
    const message = err instanceof Error ? err.message : 'Unknown error';
    console.error('[runAnalysis]', message);
    return { success: false, error: message };
  }
}
```

Langkah 5: Route Handler (Optional — Jika Butuh REST Endpoint Sendiri)

Berguna jika Anda ingin UI memanggil endpoint `/api/analyze` di Next.js terlebih dahulu sebelum diteruskan ke Reasoning Engine:

```
// app/api/analyze/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { analyzeBatch } from '@lib/reasoning-engine';

export async function POST(req: NextRequest) {
  try {
    const { assessments } = await req.json();

    if (!Array.isArray(assessments) || assessments.length === 0) {
      return NextResponse.json({ error: 'Field assessments wajib diisi dan tidak boleh kosong' }, { status: 400 });
    }

    const results = await analyzeBatch(assessments);
    return NextResponse.json(results);
  } catch (err) {
    const message = err instanceof Error ? err.message : 'Internal Server Error';
    return NextResponse.json({ error: message }, { status: 500 });
  }
}
```

Langkah 6: Contoh Komponen React (Client Component)

Contoh tombol analisis yang memanggil Server Action dari Langkah 4:

```
// components/AnalysisButton.tsx
'use client';

import { useState } from 'react';
import { runAnalysis } from '@app/actions/analyze';
import type { Assessment, AnalysisResult } from '@types/reasoning-engine';

interface Props {
  assessments: Assessment[];
}

export default function AnalysisButton({ assessments }: Props) {
  const [status, setStatus] = useState<'idle' | 'loading' | 'done' | 'error'>('idle');
  const [results, setResults] = useState<AnalysisResult[]>([]);
  const [error, setError] = useState<string | null>(null);

  async function handleClick() {
    setStatus('loading');
    setError(null);

    const res = await runAnalysis(assessments);

    if (res.success) {
      setResults(res.results);
      setStatus('done');
    } else {
      setError(res.error);
      setStatus('error');
    }
  }

  return (
    <div>
      <button onClick={handleClick} disabled={status === 'loading'}>
        {status === 'loading' ? 'Menganalisis dengan AI...' : 'Mulai Analisis'}
      </button>

      {status === 'error' && (
        <p style={{ color: 'red' }}>⚠️ Gagal: {error}</p>
      )}

      {status === 'done' && results.map((r, i) => (
        <div key={i} style={{ border: '1px solid #eee', padding: 16, marginTop: 8 }}>
          <p><strong>Status:</strong> {r.classification}</p>
          <p><strong>Analisis:</strong> {r.strength_weakness}</p>
        </div>
      ))}
    </div>
  );
}
```

```

        <ul>
          {r.action_plan.map((step, j) => <li key={j}>{step}</li>)}
        </ul>
      </div>
    )}}
  </div>
);
}

```

Cara pakai komponen di atas di halaman Anda:

```

// app/assessment/page.tsx
import AnalysisButton from '@components/AnalysisButton';

const assessmentData = [
  {
    dimension: 'Technology',
    sub_dimension: 'Information Capability',
    final_result: 2,
    expected_level: 3,
    assessment_note: 'Dashboard tersedia tapi belum ada prediksi AI',
  },
];

export default function AssessmentPage() {
  return (
    <main>
      <h1>Asesmen HP3M</h1>
      <AnalysisButton assessments={assessmentData} />
    </main>
  );
}

```

Error yang Mungkin Terjadi

HTTP Status	Penyebab	Solusi
401 Unauthorized	Header <code>X-API-Key</code> tidak ada atau salah	Pastikan nilai <code>REASONING_ENGINE_API_KEY</code> di <code>.env.local</code> sudah benar
422 Unprocessable Entity	Field wajib (<code>model</code> , <code>name</code> , dll.) bernilai <code>null</code> atau tipe data salah	Validasi semua field sebelum dikirim; gunakan fallback jika nullable
	Reasoning Engine	

500 Internal Server Error	gagal menghubungi provider LLM (API Key provider salah, model tidak ditemukan)	Periksa nilai <code>api_key</code> dan <code>model</code> di objek <code>provider</code>
Hasil <code>classification:</code> "Mixed" , kolom lain kosong	Request berhasil tapi kode mengambil data dari key JSON yang salah	Pastikan field diakses langsung: <code>result.classification</code> , bukan <code>result.reasoning_output.classification</code>